Docker & Kubernetes for Developers

Session 05: Durum Yönetimi ve Veri Kalıcılığı

Eğitmen: Hüseyin Babal
Süre: 2 Saat (Teori + Hands-on Lab)

📋 Bu Oturum İçeriği

- ✅ StatefulSets vs Deployments
- ✅ Persistent Volumes (PV) ve Persistent Volume Claims (PVC)
- ✅ Storage Classes ve dynamic provisioning
- ✅ Volume types ve kullanım alanları
- ✅ Database deployment best practices
- ✅ **E-commerce veritabanları persistent storage (Lab)**

💾 Kubernetes Storage Challenges

Container'ların Storage Problemleri:

- **Ephemeral nature** - Container restart'ta data kaybolur
- **Shared storage** - Multiple pod'lar aynı data'ya nasıl erişecek?
- **Performance** - Database'ler için fast storage gerekli
- **Backup/Recovery** - Data nasıl korunacak?
- **Scaling** - Storage dinamik olarak büyüyebilmeli

Çözüm: Kubernetes Persistent Storage

📚 Volume Types Genel Bakış

Volume Type	Açıklama	Kullanım Durumu
emptyDir	Pod ile birlikte oluşur/silinir	Temporary files, cache
hostPath	Node'un file system'ı	Development, node-specific data
persistentVolume	Cluster-level storage	Production databases
configMap	Configuration data	App configs
secret	Sensitive data	Passwords, keys
nfs	Network File System	Shared storage

📂 emptyDir vs hostPath

emptyDir Volume

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: test-pod
spec:
  containers:
  - name: app
    image: nginx
    volumeMounts:
    - mountPath: /cache
      name: cache-volume
  volumes:
  - name: cache-volume
    emptyDir: {}  # Pod restart'ta silinir
```

hostPath Volume

```yaml
volumes:
- name: host-volume
  hostPath:
    path: /data/app  # Node'un path'i
    type: DirectoryOrCreate
```

🏛️ Persistent Volume (PV) Architecture

```
┌─────────────┐    ┌─────────────┐    ┌─────────────┐
│ Application │    │    PVC      │    │    PV       │
│    Pod      │ -> │(Claim/Request)│ -> │(Actual Storage)│
└─────────────┘    └─────────────┘    └─────────────┘
                                              │
                                      ┌─────────────┐
                                      │Physical Storage│
                                      │(EBS,NFS,GCE...)│
                                      └─────────────┘
```

PV: Admin tarafından provisioned storage
PVC: Developer tarafından storage request

🎯 Persistent Volume (PV)

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: mysql-pv
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce    # Single node R/W
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  hostPath:
    path: /mnt/data/mysql
```

Access Modes:

- **ReadWriteOnce (RWO):** Single node read/write
- **ReadOnlyMany (ROX):** Multiple nodes read-only
- **ReadWriteMany (RWX):** Multiple nodes read/write

📝 Persistent Volume Claim (PVC)

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi      # PV'den 5Gi claim et
  storageClassName: manual
```

PVC Lifecycle:

1. **Pending** - Uygun PV aranıyor
2. **Bound** - PV ile bağlandı
3. **Released** - PVC silindi, PV cleanup bekliyor

🔧 Storage Classes - Dynamic Provisioning

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: kubernetes.io/aws-ebs
parameters:
  type: gp3
  replication-type: none
  zones: us-west-2a
allowVolumeExpansion: true
reclaimPolicy: Delete
```

Dynamic Provisioning Benefits:

- Otomatik PV oluşturma
- Scalable storage management
- Cloud provider integration
- Per-application storage requirements

🗃️ StatefulSets vs Deployments

Özellik	Deployment	StatefulSet
Pod Identity	Random names	Ordered (app-0, app-1)
Pod IP	Dynamic	Sticky (DNS)
Storage	Shared	Dedicated per pod
Scaling	Parallel	Sequential
Updates	Rolling	Rolling (ordered)
Use Case	Stateless apps	Databases, queues

StatefulSet Kullanım Alanları:

- Databases (MySQL, PostgreSQL, MongoDB)
- Message queues (Kafka, RabbitMQ)
- Distributed systems (Elasticsearch, Cassandra)

🏗️ StatefulSet Anatomy

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: mysql-headless  # Headless service gerekli
  replicas: 3
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
      - name: mysql
        image: mysql:8.0
        env:
        - name: MYSQL_ROOT_PASSWORD
          value: password
        volumeMounts:
        - name: data
          mountPath: /var/lib/mysql
  volumeClaimTemplates:  # Her pod için ayrı PVC
  - metadata:
      name: data
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
        requests:
          storage: 10Gi
```

🎪 Headless Service

```yaml
apiVersion: v1
kind: Service
metadata:
  name: mysql-headless
spec:
  clusterIP: None    # Headless service
  selector:
    app: mysql
  ports:
  - port: 3306
```

Headless Service Özellikleri:

- **clusterIP: None**
- **Direct pod IP resolution**
- **StatefulSet pod'ları için stable network identity**
- **DNS:** `mysql-0.mysql-headless.default.svc.cluster.local`

🧪 Lab: MySQL Database Setup

Step 1: MySQL StatefulSet

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: mysql-headless
  replicas: 1
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
      - name: mysql
        image: mysql:8.0
        env:
        - name: MYSQL_ROOT_PASSWORD
          value: rootpassword
        - name: MYSQL_DATABASE
          value: userdb
        - name: MYSQL_USER
          value: userservice
        - name: MYSQL_PASSWORD
          value: password
        ports:
        - containerPort: 3306
        volumeMounts:
        - name: mysql-data
          mountPath: /var/lib/mysql
  volumeClaimTemplates:
  - metadata:
      name: mysql-data
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
        requests:
```

🐘 Lab: PostgreSQL Setup

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgresql
spec:
  serviceName: postgresql-headless
  replicas: 1
  selector:
    matchLabels:
      app: postgresql
  template:
    metadata:
      labels:
        app: postgresql
    spec:
      containers:
      - name: postgresql
        image: postgres:15
        env:
        - name: POSTGRES_DB
          value: productdb
        - name: POSTGRES_USER
          value: productservice
        - name: POSTGRES_PASSWORD
          value: password
        ports:
        - containerPort: 5432
        volumeMounts:
        - name: postgres-data
          mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
  - metadata:
      name: postgres-data
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
        requests:
          storage: 10Gi
```

🍃 Lab: MongoDB Setup

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mongodb
spec:
  serviceName: mongodb-headless
  replicas: 1
  selector:
    matchLabels:
      app: mongodb
  template:
    metadata:
      labels:
        app: mongodb
    spec:
      containers:
      - name: mongodb
        image: mongo:7.0
        env:
        - name: MONGO_INITDB_ROOT_USERNAME
          value: root
        - name: MONGO_INITDB_ROOT_PASSWORD
          value: rootpassword
        - name: MONGO_INITDB_DATABASE
          value: orderdb
        ports:
        - containerPort: 27017
        volumeMounts:
        - name: mongo-data
          mountPath: /data/db
  volumeClaimTemplates:
  - metadata:
      name: mongo-data
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
        requests:
          storage: 10Gi
```

🌐 Lab: Headless Services

```yaml
---
apiVersion: v1
kind: Service
metadata:
  name: mysql-headless
spec:
  clusterIP: None
  selector:
    app: mysql
  ports:
  - port: 3306

---
apiVersion: v1
kind: Service
metadata:
  name: postgresql-headless
spec:
  clusterIP: None
  selector:
    app: postgresql
  ports:
  - port: 5432

---
apiVersion: v1
kind: Service
metadata:
  name: mongodb-headless
spec:
  clusterIP: None
  selector:
    app: mongodb
  ports:
  - port: 27017
```

🔍 Lab: Database Connection Test

MySQL Test

```
kubectl run mysql-client --image=mysql:8.0 --rm -it -- mysql -h mysql-0.mysql-headless -u userservice -p
# Password: password
```

PostgreSQL Test

```
kubectl run postgres-client --image=postgres:15 --rm -it -- psql -h postgresql-0.postgresql-headless -U productservice -d productdb
# Password: password
```

MongoDB Test

```
kubectl run mongo-client --image=mongo:7.0 --rm -it -- mongosh mongodb://root:rootpassword@mongodb-0.mongodb-headless:27017/orderdb
```

📊 Storage Monitoring

PVC Status Kontrolü

```
kubectl get pvc
kubectl describe pvc mysql-data-mysql-0

# PV bilgileri
kubectl get pv
kubectl describe pv <pv-name>
```

Storage Usage

```
# Pod içindeki disk kullanımı
kubectl exec mysql-0 -- df -h /var/lib/mysql

# Storage class'ları
kubectl get storageclass
```

💡 Database Best Practices

Resource Limits

```yaml
resources:
  requests:
    memory: "512Mi"
    cpu: "500m"
  limits:
    memory: "1Gi"
    cpu: "1000m"
```

Readiness & Liveness Probes

```yaml
readinessProbe:
  exec:
    command:
    - mysqladmin
    - ping
    - -h
    - localhost
  initialDelaySeconds: 30
  periodSeconds: 10

livenessProbe:
  exec:
    command:
    - mysqladmin
    - ping
    - -h
```

📝 Sonraki Oturum Önizlemesi

Session 06: Konfigürasyon Yönetimi ve Sırlar

- **ConfigMaps** ve environment variables
- **Secrets** ve sensitive data yönetimi
- **Configuration injection** patterns
- **Security best practices**
- **E-commerce microservices** configuration management

Hazırlık:

- Storage kavramlarını pekiştirin
- Configuration management teorisi

🙏 Teşekkürler!

Storage Expert! 💾

Bugün öğrendiklerimiz:
✅ StatefulSets ve Deployments farkını öğrendik
✅ Persistent Volume architecture'ını anladık
✅ Database'leri Kubernetes'te deploy ettik
✅ Storage best practices öğrendik

İletişim:

- GitHub: @huseyinbabal

Sonraki Oturum: Session 06 - Konfigürasyon Yönetimi ve Sırlar